6.7920: Reinforcement Learning **Lecture Date: October 6, 2026**

L08. Model-Free Policy Evaluation

Lecturer: Cathy Wu *Scribe: Jinhee Won*

1 Introduction

So far, we have studied dynamic programming, where the reward function and transition model are known. However, in reinforcement learning, we may not have access to either. Instead, we can interact with the environment and observe samples of rewards and transitions.

This lecture focuses on the model-free setting: how to evaluate a policy without explicitly learning a model of the reward or transition dynamics.

2 Reinforcement Learning vs. Dynamic Programming

So far, we have focused on dynamic programming, where we have direct access to the reward function r and transition model P. In reinforcement learning, we instead consider settings where these components of the MDP are unknown. Rather than evaluating them directly, we interact with the environment and observe samples of their outputs.

As a motivating example, consider the Atari game *Breakout*. An agent observes the game state, chooses actions, and receives rewards based on what happens next. The agent does not need direct access to the game's transition or reward model; it can learn from interaction.

Today we look at the class of *model-free* methods: methods that learn directly from interaction without explicitly constructing a reward or transition model.

Aside

Reinforcement learning comes with several different distinctions, including model-free vs. model-based, online vs. offline, and on-policy vs. off-policy methods. These are all different terms that we will cover in this and future lectures, and a good framework to identify and think about RL algorithms.

3 Model-free Policy Evaluation

Recall that policy iteration alternates between two steps: *policy evaluation* and *policy improvement* (these are also referred to as *prediction* and *control*, respectively).

In this lecture, we focus on policy evaluation: given a fixed policy π, we want to estimate its value function

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t r(s_t, \pi(s_t)) \,\middle|\, s_0 = s; \pi\right].$$

Aside

For the remaining lectures, we will work in the episodic discounted setting unless stated otherwise.

Ultimately, our goal in reinforcement learning is to find a policy that maximizes cumulative reward over time. Here, we take one step toward that goal by first learning how to evaluate a fixed policy from interaction (not known transition or reward function) with the environment.

3.1 The RL Interaction Protocol

In reinforcement learning, we do not have direct access to the reward function or transition model. Instead, we interact with the environment and observe samples of rewards and transitions.

For a fixed policy π, we can generate multiple *trajectories*, also called *episodes* or *rollouts*. For rollout i, we observe a sequence

$$s_0,\ a_0^{(i)},\ r_1^{(i)},\ s_1^{(i)},\ a_1^{(i)},\ r_2^{(i)},\ \ldots,\ s_T^{(i)},$$

where

$$a_t^{(i)} = \pi(s_t^{(i)}).$$

Repeating this interaction gives us different sampled trajectories under the same policy. We will use these samples to estimate the value function.

4 Monte-Carlo Policy Evaluation

One way to estimate the value function is to generate multiple rollouts, compute the return from each rollout, and average them.

For rollout i, the return starting from s_0 is

$$\hat{R}_i(s_0) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}^{(i)},$$

where i indexes the rollout. The Monte Carlo estimate of the value function is then

$$\hat{V}_n^{\pi}(s_0) = \frac{1}{n} \sum_{i=1}^{n} \hat{R}_i(s_0).$$

That is, we estimate the expected return under policy π by taking the empirical average of the returns observed across n trajectories. This is called *Monte Carlo policy evaluation*.

Algorithm 1 Monte Carlo Policy Evaluation

Input: fixed policy π
for $i = 1, \ldots, n$ **do**
  Set $t = 0$
  Set initial state s_0
  while s_t is not terminal **do**
    Take action $a_t = \pi(s_t)$
    Observe next state s_{t+1} and reward r_{t+1}
    $t \leftarrow t + 1$
  end while
  Compute return

$$\hat{R}_i(s_0) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}^{(i)}$$

end for
Estimate

$$\hat{V}_n^\pi(s_0) = \frac{1}{n} \sum_{i=1}^{n} \hat{R}_i(s_0)$$

4.1 Theoretically

Monte Carlo policy evaluation is an instance of Monte Carlo approximation of a mean. Let X be a random variable with mean

$$\mu = \mathbb{E}[X],$$

and let $x_1, \ldots, x_n$ be i.i.d. samples from X. The Monte Carlo estimate of the mean is

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

In our setting, the random variable is the return from a trajectory. Since each return $\hat{R}_i(s_0)$ is an unbiased estimator of $V^\pi(s_0)$,

$$\mathbb{E}\left[\hat{R}_i(s_0)\right] = V^\pi(s_0).$$

Therefore,

$$\hat{V}_n^\pi(s_0) = \frac{1}{n} \sum_{i=1}^{n} \hat{R}_i(s_0)$$

is simply the empirical mean of the observed returns.

By the strong law of large numbers,

$$\hat{V}_n^\pi(s_0) \xrightarrow{\text{a.s.}} V^\pi(s_0) \qquad \text{as } n \to \infty.$$

Aside

For an infinite-horizon problem, we can truncate each trajectory after H steps and use

$$\hat{R}_i^{(H)}(s_0) = \sum_{t=0}^{H-1} \gamma^t r_{t+1}^{(i)}.$$

Although we ignore the return after step H, given $\gamma < 1$, the contribution of those later rewards decays geometrically. If rewards are bounded by $r_{\max}$, then

$$|V_H^\pi(s_0) - V^\pi(s_0)| \leq \frac{\gamma^H r_{\max}}{1-\gamma}.$$

Thus, for sufficiently large H, the truncation error becomes small.

4.2 Why can't we just stop here?

The Monte Carlo estimator above is a *batch* computation: we first collect the returns from our trajectories and then average them to compute $\hat{V}_n^\pi$. However, we might want to update our estimate incrementally as new trajectories arrive, without recomputing the entire average each time. This motivates an *incremental* version of Monte Carlo policy evaluation.

4.3 Incremental Monte-Carlo Policy Evaluation

We can rewrite the Monte Carlo estimate incrementally. Recall that the return of trajectory i starting from s_0 is

$$\hat{R}_i(s_0) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}^{(i)}.$$

The estimated value function after n trajectories is

$$\hat{V}_n^\pi(s_0) = \frac{1}{n} \sum_{i=1}^{n} \hat{R}_i(s_0).$$

This can be rewritten as

$$\hat{V}_n^\pi(s_0) = \frac{n-1}{n} \hat{V}_{n-1}^\pi(s_0) + \frac{1}{n} \hat{R}_n(s_0)$$

and more generally,

$$\hat{V}_n^\pi(s_0) = (1-\eta_n)\, \hat{V}_{n-1}^\pi(s_0) + \eta_n \hat{R}_n(s_0),$$

where η_n is the learning rate. This lets us compute a running average!

Algorithm 2 Incremental Monte Carlo Policy Evaluation

Input: fixed policy π
for $i = 1, \ldots, n$ **do**
    Generate one trajectory under π
    Compute the return $\hat{R}_i(s_0)$
    Update

$$\hat{V}_i^\pi(s_0) \leftarrow (1-\eta_i)\hat{V}_{i-1}^\pi(s_0) + \eta_i \hat{R}_i(s_0)$$

end for

4.4 Learning rate Conditions

So far, the incremental Monte Carlo update used

$$\eta_n = \frac{1}{n},$$

which exactly recovers the empirical average. More generally, we can use other learning-rates as long as they satisfy the Robbins–Monro conditions:

$$\sum_{n=1}^{\infty} \eta_n = \infty, \qquad \sum_{n=1}^{\infty} \eta_n^2 < \infty.$$

The first condition ensures that the updates do not decay too quickly: even far into learning, new samples must still have enough total influence to correct the estimate.

The second condition requires the steps to shrink sufficiently fast so that the noise from sampled returns is eventually controlled.

Under these conditions,

$$\hat{V}_n^{\pi}(s_0) \xrightarrow{\text{a.s.}} V^{\pi}(s_0).$$

We will discuss further during recitation.

Aside

Why would we want to use a different learning rate schedule? Practically, we might find that some learning rates do better or worse compared to others (as seen in ML algorithms). Theoretically, we want the most general form if possible that guarantees convergence.

5 Temporal-Difference, TD(λ)

Temporal-difference methods update value estimates using the difference between two estimates at different points in time. This difference is called the *temporal-difference error*. The main idea is that different TD methods differ in *when* they update the value estimate and how far into the future they look before forming that update.

5.1 TD(1)

Incremental Monte Carlo can also be viewed as $TD(1)$. The update uses the full return from a trajectory, so information is propagated across the entire rollout before updating the value estimate.

5.2 TD(0)

Unlike incremental Monte Carlo, $TD(0)$ updates the value estimate *within* a trajectory. The key idea is to use the recursive Bellman equation

$$V^{\pi}(s) = r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,\pi(s))} \left[V^{\pi}(s') \right].$$

In the model-free setting, we do not know the reward or transition model directly, but we can replace these quantities with observed samples. After observing a transition

$$(s_t, a_t, r_{t+1}, s_{t+1}),$$

we update

$$\hat{V}^\pi(s_t) \leftarrow (1-\eta)\hat{V}^\pi(s_t) + \eta\left[r_{t+1} + \gamma\hat{V}^\pi(s_{t+1})\right],$$

which we can rewrite as

$$\hat{V}^\pi(s_t) \leftarrow \hat{V}^\pi(s_t) + \eta\underbrace{\left(r_{t+1} + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right)}_{\delta_t}.$$

We can similarly show convergence using the Robbins–Monro conditions from incremental Monte Carlo (if every state is visited infinitely often).

Aside

We connect this to the Bellman error. For a value estimate $\hat{V}^\pi$, the Bellman error at state s is

$$\mathcal{B}^\pi(\hat{V}^\pi; s) = \mathcal{T}^\pi\hat{V}^\pi(s) - \hat{V}^\pi(s).$$

Conditioned on s_t, the TD error δ_t is an unbiased sample of this Bellman error:

$$\mathbb{E}[\delta_t \mid s_t] = \mathcal{B}^\pi(\hat{V}^\pi; s_t).$$

Thus, each TD update uses a sampled estimate of how far the current value estimate is from satisfying the Bellman equation.

5.3 Defining TD

We now formally define TD. The temporal-difference error for a transition

$$(s_t, r_{t+1}, s_{t+1})$$

is

$$\delta_t = r_{t+1} + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t).$$

For incremental Monte Carlo, or $TD(1)$,

$$\hat{V}^\pi(s_0) \leftarrow \hat{V}^\pi(s_0) + \eta\left(\delta_0 + \gamma\delta_1 + \cdots + \gamma^{T-1}\delta_{T-1}\right).$$

Because it uses the entire trajectory, $TD(1)$ has no bootstrapping bias but can have high variance for long trajectories.

In contrast, $TD(0)$ uses only the immediate TD error,

$$\hat{V}^\pi(s_0) \leftarrow \hat{V}^\pi(s_0) + \eta\delta_0.$$

This has lower variance, but introduces bias by bootstrapping from the current estimate of the next state's value.

This naturally raises the question: can we choose something between $TD(0)$ and $TD(1)$, such as $TD(0.5)$ that would control the bias-variance tradeoff?

5.4 TD(λ)

We can interpolate between $TD(0)$ and $TD(1)$ by using a weighted combination of future temporal-difference errors. For a state s_t,

$$\hat{V}^\pi(s_t) \leftarrow \hat{V}^\pi(s_t) + \eta(s_t) \sum_{k=t}^{T-1} (\gamma\lambda)^{k-t} \delta_k.$$

When $\lambda = 0$, only the immediate TD error is used, giving $TD(0)$. As λ approaches 1, more of the future trajectory is included, approaching incremental Monte Carlo, or $TD(1)$.

We can also define the corresponding $TD(\lambda)$ Bellman operator:

$$\mathcal{T}_\lambda^\pi = (1-\lambda) \sum_{m=0}^{\infty} \lambda^m (\mathcal{T}^\pi)^{m+1},$$

which is a weighted combination of multi-step Bellman operators, and it retains the same contraction and convergence properties.

5.5 How do we compute TD(λ)?

However, by reconstructing our updates as such, we again run into the issue of updating our value function less frequently (at the end of each trajectory, not transition). To fix this issue, we can use *eligibility traces.* Eligibility traces act as a short-term memory that records how much each previously visited state should be affected by the current TD error.

Let

$$z(s)$$

denote the eligibility trace for state s. At the beginning of each episode,

$$z(s) = 0 \qquad \text{for all } s.$$

For each transition, we first compute

$$\delta_t = r_{t+1} + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t).$$

Then update the eligibility traces:

$$z(s) \leftarrow \begin{cases} \gamma\lambda z(s) + 1, & s = s_t, \\ \gamma\lambda z(s), & s \neq s_t. \end{cases}$$

Finally, update every state:

$$\hat{V}^\pi(s) \leftarrow \hat{V}^\pi(s) + \eta(s)\, z(s)\, \delta_t.$$

The intuition is that recently visited states have larger eligibility traces, so they receive a larger share of the current TD error. On the other hand, older states gradually "fade out" by the factor $\gamma\lambda$.